

RAPORT INDYWIDUALNY

AirSense Weather AI

Szczegółowy opis wkładu każdego członka zespołu (Grupa 5)

Projekt	AirSense Weather AI — system predykcji jakości powietrza i pogody
Grupa	Grupa 5
Autorzy	Jakub Moszyński, Damian Tomczyk, Nikol Olszewska, Łukasz Duss
Technologie	Python, Streamlit, TensorFlow/Keras (LSTM), scikit-learn, Plotly
Źródła danych	GIOŚ API v1, Open-Meteo API
Data	3 czerwca 2026

1. Zespół

Projekt AirSense Weather AI zrealizował czteroosobowy zespół. Praca została podzielona zgodnie z rolami w typowym projekcie ML — od inżynierii danych, przez analizę i model, po integrację i ewaluację. Poniższa tabela podsumowuje role, a kolejne sekcje opisują indywidualny wkład każdej osoby.

Osoba	Rola	Główny obszar
Jakub Moszyński	Data Engineer	Pobieranie i czyszczenie danych, sekwencje
Damian Tomczyk	Data Analyst	EDA, dashboard i wizualizacje
Nikol Olszewska	Deep Learning Engineer	Architektura i trening modelu LSTM
Łukasz Duss	ML Analyst / Project Manager	Integracja, ewaluacja, rejestr modeli

2. Wkład indywidualny

2.1. Jakub Moszyński — Data Engineer

Zakres odpowiedzialności: integracja z zewnętrznymi API, pozyskanie surowych danych i ich wstępne przygotowanie do dalszej analizy.

Kluczowy wkład:

- Implementacja funkcji **fetch_real_climate_data** łączącej dwa źródła (Open-Meteo i GIOŚ).
- Pobieranie danych godzinowych z Open-Meteo i agregacja do wartości dziennych.
- Obsługa GIOŚ API v1: nagłówki HTTP imitujące przeglądarkę oraz elastyczne parsowanie polskich kluczy odpowiedzi („Wskaźnik - kod”, „Wartość”, „Identyfikator stanowiska”) i pobieranie pomiarów per sensor.
- Czyszczenie danych: interpolacja liniowa braków oraz funkcja **remove_outliers** (prycinanie do 1./99. percentyla).
- Przygotowanie sekwencji czasowych (**prepare_sequences**) jako wejścia do modelu.

Wyzwania i decyzje: serwery GIOŚ odrzucały zapytania bez nagłówków — rozwiązaniem było dołączenie User-Agent i Referer oraz odporna na różne formaty obsługa JSON. Braki i błędne piki rozwiązano interpolacją i winsoryzacją zamiast usuwania rekordów.

Rezultat: spójny, dzienny zbiór danych gotowy do analizy i treningu, budowany na żywo z publicznych API.

2.2. Damian Tomczyk — Data Analyst

Zakres odpowiedzialności: analiza eksploracyjna danych (EDA) oraz warstwa wizualizacji w interfejsie.

Kluczowy wkład:

- Zakładka „EDA i Trendy”: wybór parametru i interaktywny wykres odczytów w Plotly.

- Obliczanie 24-godzinnej średniej kroczącej (trend dobowy) jako osobnej serii (bez zmiany liczby cech modelu).
- Dodanie pionowej linii z dzisiejszą datą („Dziś”) oddzielającej historię od prognozy.
- Czytelne formatowanie wykresów (ciemny motyw, hovermode), spójne z resztą aplikacji.

Wyzwania i decyzje: trend liczony jest poza macierzą wejściową modelu, co rozwiązuje problem niezgodności wymiarów. Po uwadze recenzenta usunięto pobieranie dni „w przyszłość”, dzięki czemu linia „Dziś” trafia na koniec realnej historii.

Rezultat: przejrzysty dashboard, który pozwala szybko zrozumieć trendy i sezonowość danych dla wybranej stacji.

2.3. Nikol Olszewska — Deep Learning Engineer

Zakres odpowiedzialności: projekt, implementacja i trening sieci neuronowej.

Kluczowy wkład:

- Architektura LSTM typu sequence-to-sequence: dwie warstwy LSTM, Dropout 0.2, warstwa Dense i Reshape.
- Skalowanie danych (MinMaxScaler) oraz zapis parametrów skalera do późniejszej inferencji.
- Dynamiczna liczba cech — model dopasowuje kształt wejścia/wyjścia do liczby sensorów stacji.
- Konfiguracja treningu (Adam, MSE) i integracja paska postępu epok w interfejsie (callback Keras).

Wyzwania i decyzje: przy stosunkowo małym zbiorze zastosowano Dropout, aby ograniczyć przeuczenie. Architektura Seq2Seq pozwala prognozować kilka dni naprzód dla wszystkich cech jednocześnie.

Rezultat: elastyczny model, który trenuje się od nowa dla dowolnej stacji i dowolnej liczby dostępnych parametrów.

2.4. Łukasz Duss — ML Analyst / Project Manager

Zakres odpowiedzialności: integracja całej aplikacji, ewaluacja modelu oraz zarządzanie projektem i przepływem danych między zakładkami.

Kluczowy wkład:

- Spięcie etapów w jedną aplikację Streamlit (zakładki EDA / trening / predykcja).
- Chronologiczny podział zbioru 80 / 10 / 10 (trening / walidacja / test).
- Ewaluacja modelu: MAE, MSE, RMSE i R^2 liczone w jednostkach rzeczywistych oraz w rozbiciu na cechy.
- Rejestr modeli: zapis i odczyt modeli oraz metadanych (_meta.npy), wybór modelu do predykcji.
- Utrzymywanie wyników w Session State, odtwarzanie skalera i generowanie prognozy na realne daty.

Wyzwania i decyzje: metryki przeniesiono ze skali [0, 1] do jednostek rzeczywistych, co uczyniło je interpretowalnymi. Zabezpieczono odtwarzanie skalera przed cechami stałymi (brak prognoz NaN). Session State rozwiązuje problem znikania wyników po każdej interakcji w Streamlit.

Rezultat: kompletna, działająca aplikacja end-to-end z czytelnym raportem ewaluacji i powtarzalną predykcją.

3. Macierz wkładu

Oznaczenia: ● — odpowiedzialność główna, ○ — wsparcie.

Etap	Jakub	Damian	Nikol	Łukasz
Pobieranie danych (API)	●			
Czyszczenie / outliery	●			
Sekwencje czasowe	●		○	
EDA / wizualizacje		●		
Architektura LSTM			●	
Skalowanie danych			●	○
Trening + podział 80/10/10			●	●
Ewaluacja (metryki)			○	●
Predykcja / inferencja		○		●
Integracja i UI		○		●
Rejestr modeli / Session State				●

4. Wspólne usprawnienia jakości

Po recenzji zespół wprowadził poprawki, które dotknęły kilku obszarów odpowiedzialności:

- Naprawa parsera GIOŚ v1 (polskie klucze) — bez niej brakowało wszystkich wskaźników zanieczyszczeń (widoczna była tylko pogoda) — Jakub.
- Przejście na modelowanie godzinowe zamiast dziennego (realnie trenowalny zbiór) — Nikol, Jakub.
- Linia „Dziś” i rezygnacja z godzin „w przyszłość” — Damian (wizualizacja) i Jakub (dane).
- Metryki w jednostkach rzeczywistych i w rozbiciu na cechy + early stopping — Łukasz, Nikol.
- Ciągła trajektoria/predykcja vs rzeczywistość oraz zabezpieczenie skalera — Łukasz, Nikol.